

Lotus Effect Optimization-Guided Multi-Objective Recommendation for QMOF Materials Discovery

First manuscript draft with implementation, evaluation, figures, and reproducibility appendix

System	QMOF-Rec
Contribution	Lotus Effect Algorithm reranking
Dataset artifact	20,372 QMOF vector metadata records
Evaluation	Five query scenarios, six ranking methods, top-5 recommendations
Draft status	Initial manuscript draft for revision and extension

Abstract

Metal-organic frameworks provide a large design space for adsorption, catalysis, storage, and electronic applications. The size and heterogeneity of the QMOF search space make manual screening difficult, especially when a researcher must balance semantic intent, electronic suitability, density, stability, and novelty.

This manuscript introduces a Lotus Effect Algorithm (LEA)-guided recommendation layer for QMOF-Rec, a retrieval-augmented materials discovery system. The proposed system retrieves a candidate pool from vector metadata and reranks candidates using a population-based optimizer inspired by lotus mutation and self-cleaning.

The first evaluation used 20372 QMOF metadata records, five query scenarios, six ranking methods, and top-5 recommendation lists. LEA achieved mean NDCG@K of 0.996, remained close to the highest weighted relevance baseline, and produced substantially higher diversity than weighted-sum ranking.

Keywords

QMOF; metal-organic frameworks; recommender systems; Lotus Effect Algorithm; multi-objective optimization; retrieval-augmented generation; materials discovery; explainable AI.

1. Introduction

Materials discovery increasingly depends on computational databases, learned representations, and interactive decision support. MOFs are especially challenging because a single application can depend on multiple coupled properties, including pore structure, density, stability, electronic band gap, topology, and chemical composition.

QMOF-Rec addresses this challenge by combining retrieval, property scoring, graph-aware representations, and conversational scientific assistance. The present work adds a new optimization contribution: a LEA-guided reranking stage that treats recommendation as a multi-objective search over retrieved QMOF candidates.

The core idea is to use retrieval for recall and LEA for decision quality. Retrieval produces a candidate pool related to the user query. Objective scorers transform each candidate into a normalized profile. LEA then searches this profile space, preserving candidates that balance query relevance and physicochemical suitability while periodically replacing weak solutions through self-cleaning.

Contributions

- 1 An end-to-end QMOF-AI platform for materials discovery, integrating retrieval, graph learning, property prediction, scientific assistance, and recommendation.
- 2 A LEA-guided post-retrieval multi-objective reranking strategy for QMOF recommendation.
- 3 A graph-aware recommendation extension based on GraphSAGE/GAT representations.
- 4 A retrieval-augmented scientific assistant layer for evidence-grounded explanation and interaction.
- 5 A reproducible evaluation framework for ranking quality, with a clear protocol for future graph-model and LLM evaluation.

2. End-to-End QMOF-AI Platform Architecture

This manuscript presents not only LEA-based reranking, but also a broader QMOF-AI platform for materials discovery. The platform integrates retrieval-augmented scientific assistance, graph-aware representation learning, property prediction, and multi-objective recommendation for QMOF discovery.

Figure 1 is a newly constructed overview figure of the proposed platform. It is not a reused uploaded platform image. The figure was redrawn from the updated system description so that the manuscript architecture is internally consistent with the current backend, frontend, retrieval, graph, recommendation, infrastructure, and feedback-loop design.

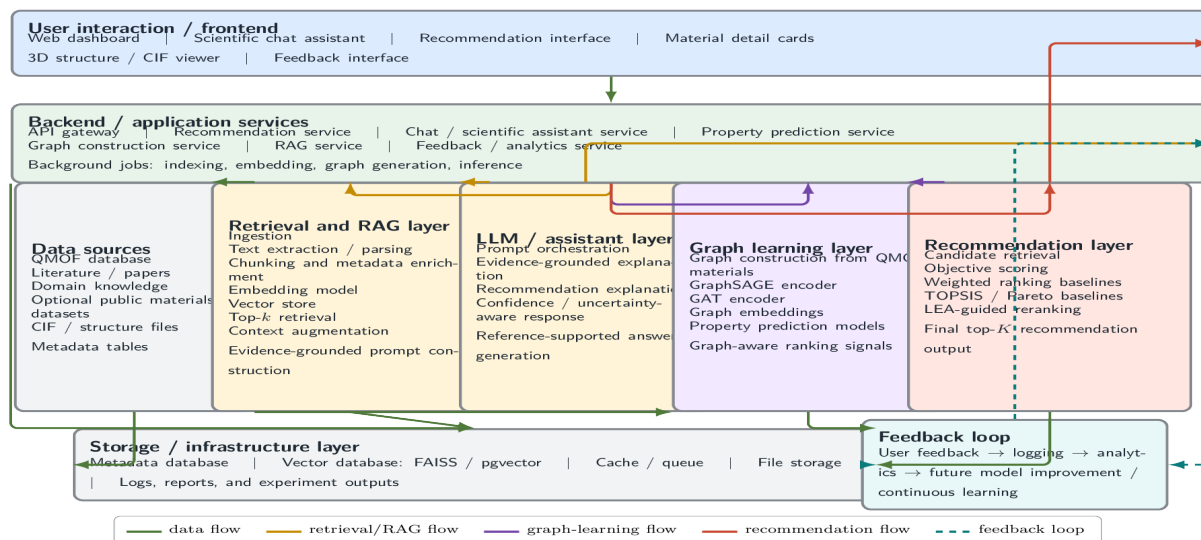


Figure 1. Overview of the end-to-end QMOF-AI platform for materials discovery, integrating frontend interaction, backend services, retrieval-augmented scientific assistance, graph learning, property prediction, multi-objective recommendation, and user feedback.

Module Status and Scope

The recommendation module is the component most directly evaluated in this draft. Candidate retrieval, objective scoring, weighted baselines, TOPSIS/Pareto baselines, LEA-guided reranking, and final top-K output are implemented in the evaluation workflow.

The graph-learning path is architecture-supported and partially implemented through graph construction and GraphSAGE-related components. GAT benchmarking, graph-embedding similarity, and complete graph-aware recommendation evaluation remain planned extensions.

The RAG and scientific assistant layer is implemented at the service level, but LLM faithfulness, citation quality, uncertainty calibration, and user-facing answer quality are not fully benchmarked in this draft. The manuscript defines a protocol for future LLM evaluation without fabricating results.

- Fully evaluated in this draft: LEA-guided ranking quality against deterministic and multi-objective baselines.
- Implemented but not fully benchmarked: frontend interaction, RAG service, CIF/structure handling, graph construction, and GraphSAGE-style prediction support.
- Planned extensions: full GAT evaluation, graph-aware ranking ablations, LLM faithfulness evaluation, user feedback learning, and expert review.

3. Background and Related Work

The QMOF database provides quantum-chemical properties for a large collection of experimentally derived and hypothetical MOFs. Such datasets enable screening, supervised property prediction, and retrieval-based exploration. However, database scale alone does not solve the ranking problem.

Retrieval-augmented generation and vector search provide flexible access to textual and metadata representations. Vector search can efficiently retrieve candidates, but retrieval distance is not identical to material suitability. A retrieved material may be semantically similar but poor in density or electronic profile.

Multi-criteria decision methods such as TOPSIS and Pareto ranking are common ways to reason about trade-offs. Weighted sums are simple and interpretable, but they can over-concentrate recommendations around one narrow optimum. Pareto approaches are more faithful to trade-offs but still need ranking inside fronts.

The LEA paper introduces a nature-inspired evolutionary optimizer based on lotus pollination, dragonfly-inspired exploration, and self-cleaning behavior. In this manuscript, the implemented version follows the simplified LEA code variant supplied with the project.

4. System Architecture

The system is organized as a modular web application. The backend exposes FastAPI routes for recommendation, chat, material prediction, and structure retrieval. The frontend is a React workspace that displays query controls, dynamic weights, recommendations, explanations, and 3D structure viewing.

The recommendation pipeline begins with user intent. Dynamic weights are generated from the query, candidates are retrieved from the FAISS vector store, physicochemical scores are computed, and a final optimizer reranks candidates.

Frontend	React/Vite workspace with recommendation panel, metric display, chat, and material cards.
Backend API	FastAPI routes for recommendation, chat, material prediction, and structure retrieval.
Retrieval layer	Sentence-transformer embeddings and FAISS vector index over QMOF metadata.
Scoring layer	Semantic, band-gap, density, porosity, stability, and similarity scores.
Optimization layer	LEA reranking over normalized objective vectors.
Evaluation layer	Standalone benchmark script producing CSV metrics and manuscript figures.

5. LEA-Guided Recommendation Method

The recommendation adaptation of LEA treats each individual as a point in normalized objective space. The dimensions are semantic relevance, band-gap suitability, density suitability, porosity, and stability.

Since candidates are fixed database records, continuous individuals are mapped to the nearest candidate objective vector during evaluation. This preserves database validity: every returned recommendation corresponds to an existing material record.

Fitness combines three terms: query-weighted suitability, objective balance, and diversity relative to already selected high-quality candidates. This design encourages LEA to find candidates that are relevant, balanced, and not redundant.

Algorithm Summary

Input	Retrieved candidate pool C , objective weights w , population size N , iterations T , top- k K .
Initialize	Seed population with candidate objective vectors and random normalized vectors.
Evaluate	Map every individual to its nearest QMOF objective vector and compute LEA fitness.
Mutation	Move each individual toward the best solution using a decaying lotus factor and random local perturbation.
Selection	Accept mutants only when their fitness improves the current individual.
Self-cleaning	Every tenth iteration, replace the weakest one-fifth of the population with fresh random solutions.
Output	Return unique QMOFs ranked by LEA fitness with LEA score, rank, and optimization metadata.

6. Data and Property Coverage

The current experiment used 20372 records from backend/vector_db/metadata.json. The full QMOF CSV was not present, so the evaluation did not assume unavailable descriptor columns.

The available metadata contains density for all materials, band gap for approximately 53.1 percent of materials, and no populated void-fraction values. This directly affects porosity and balance metrics.

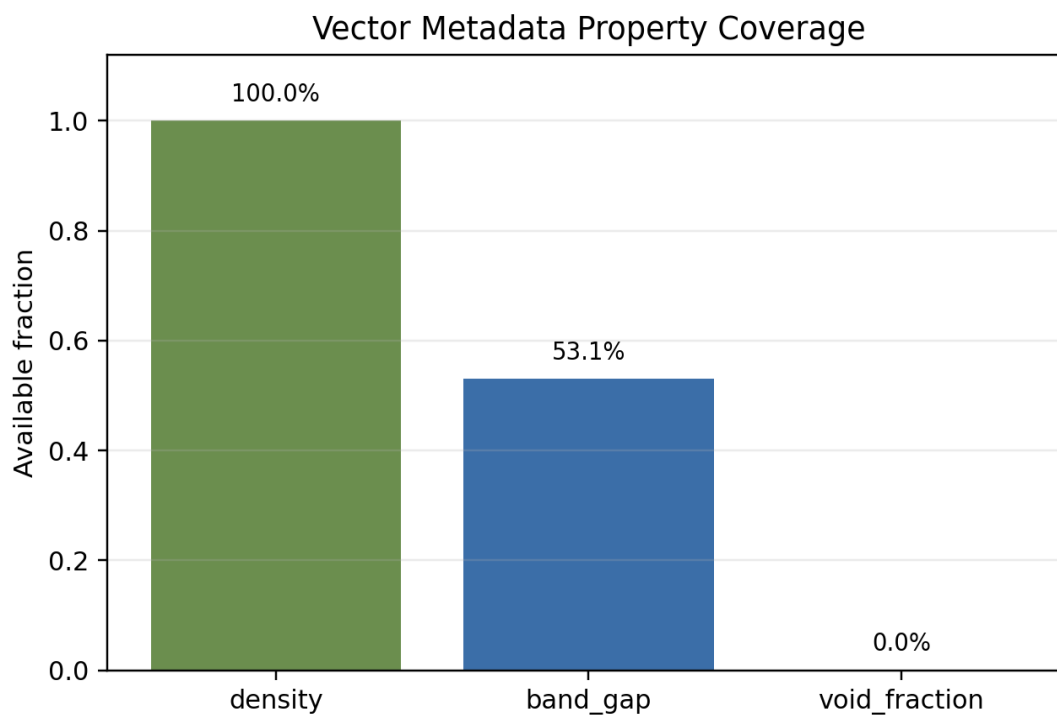


Figure 2. Property coverage in the current vector metadata.

7. Experimental Design

The benchmark evaluates ranking quality across five query scenarios selected to represent common MOF discovery intents: CO₂ adsorption, photocatalysis, lightweight gas storage, balanced materials discovery, and wide-band-gap insulating frameworks.

Each query defines a deterministic objective weight vector. A candidate pool of 100 materials is constructed using a deterministic lexical and property proxy. This design avoids external API calls or model downloads, making the experiment reproducible in the current workspace.

- SemanticOnly ranks by semantic relevance proxy.
- WeightedSum ranks by the query-weighted objective average.
- TOPSIS ranks by closeness to the ideal objective vector.
- ParetoCrowding applies non-dominated sorting with crowding and weighted tie-breaking.
- Random provides a stochastic lower reference.
- LEA applies population-based lotus mutation and self-cleaning reranking.

8. Aggregate Results

WeightedSum achieved the highest mean relevance (0.696), as expected because the metric is directly aligned with weighted scoring. LEA achieved mean relevance of 0.694, very close to the weighted optimum.

LEA achieved NDCG@K of 0.996. Its main advantage is diversity: LEA produced mean diversity of 0.176, compared with 0.017 for WeightedSum.

method	mean_relevance	diversity	ndcg_at_k	hypervolume_proxy	runtime_ms
WeightedSum	0.6961	0.0169	1.0000	3.46e-10	0.12
LEA	0.6939	0.1764	0.9959	4.24e-10	127.46
TOPSIS	0.6936	0.0527	0.9952	4.79e-10	0.13
ParetoCrowding	0.6932	0.1831	0.9951	4.15e-10	34.15
Random	0.6830	0.0522	0.9746	3.01e-10	0.07
SemanticOnly	0.6818	0.0330	0.9702	3.04e-10	0.07

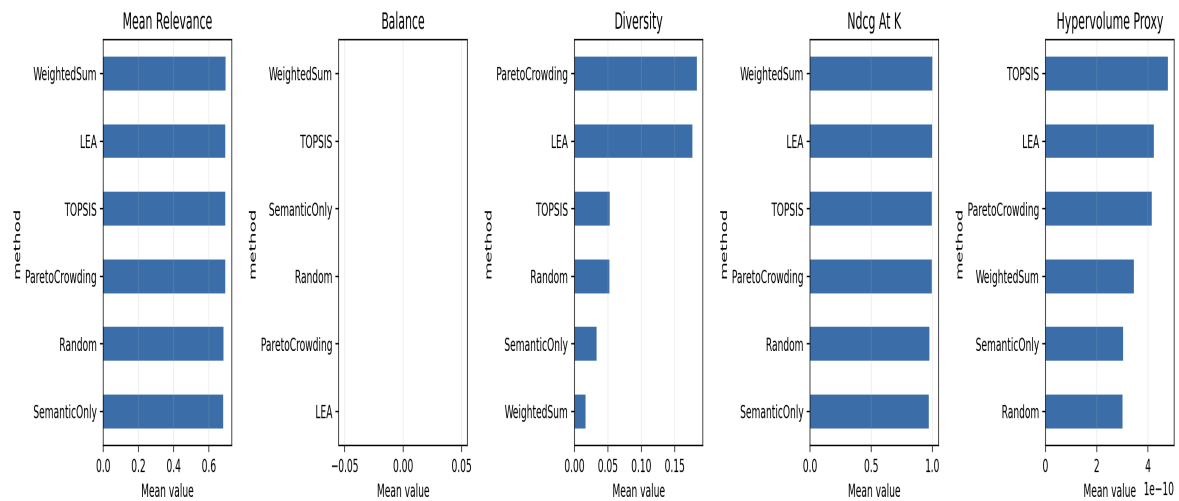


Figure 3. Aggregate comparison across ranking methods.

9. Runtime and Computational Cost

The optimization methods differ substantially in runtime. WeightedSum, SemanticOnly, TOPSIS, and Random are very fast in this benchmark. ParetoCrowding is slower because of pairwise dominance comparisons. LEA is the slowest method because it performs iterative population evaluation.

This runtime is acceptable for interactive recommendation when candidate pools are modest, but it should be optimized for larger-scale search. Potential improvements include vectorized fitness caching, adaptive early stopping, lower population size for exploratory UI queries, and full-size LEA only when the user requests optimization-grade recommendations.

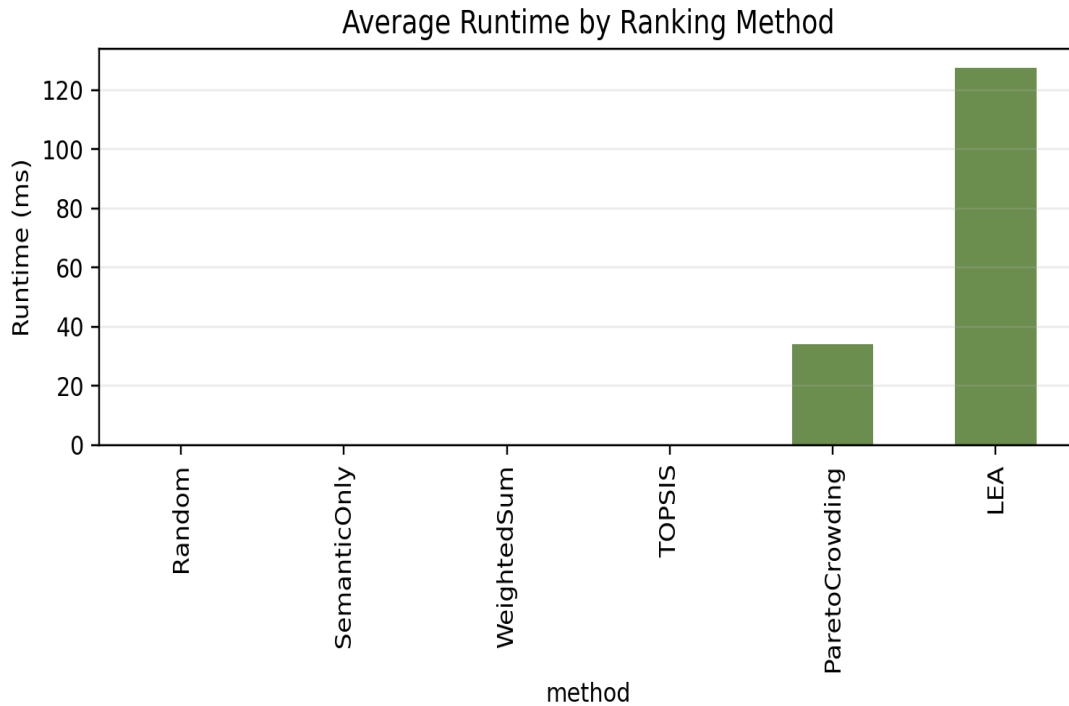


Figure 4. Average runtime by ranking method.

10. LEA Convergence

LEA convergence curves show rapid early improvement followed by stable plateaus. This behavior is expected because the candidate pool is discrete and the population is seeded with candidate objective vectors.

The convergence behavior suggests that the current 60 iterations may be more than necessary for some queries. A future adaptive version can stop when best fitness does not improve for a fixed patience window.

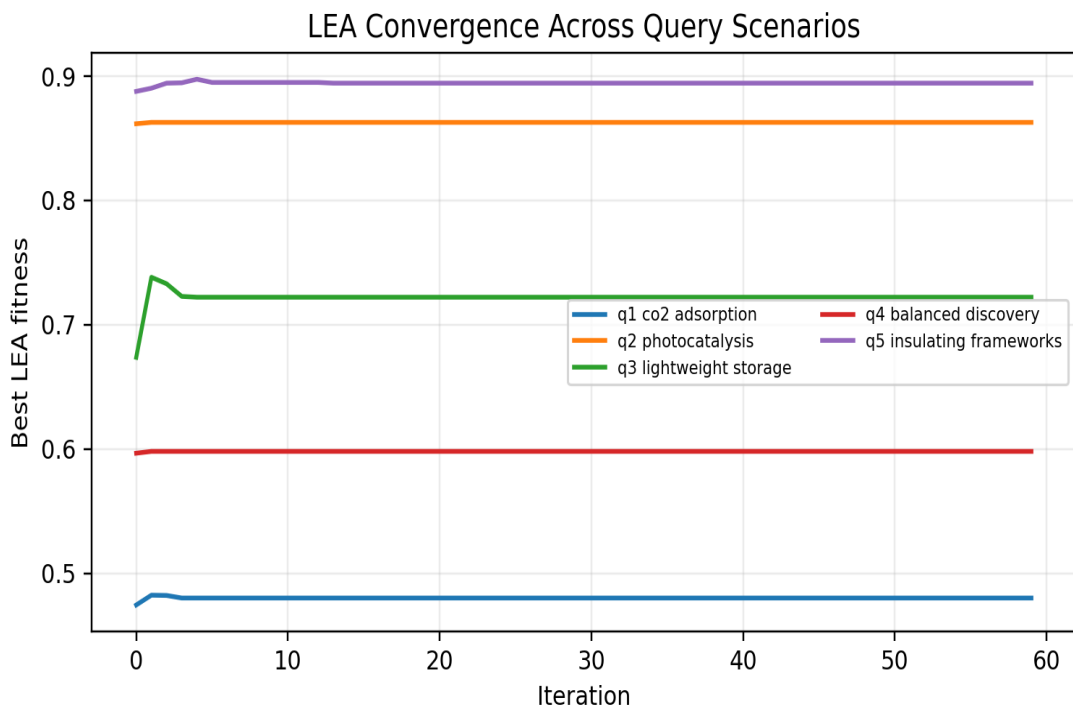


Figure 5. Best LEA fitness over 60 iterations for five query scenarios.

11. Objective Profiles

The objective radar plot compares the mean profile of LEA and WeightedSum recommendations. LEA improves band-gap suitability relative to WeightedSum while maintaining similar semantic, density, and stability profiles.

Porosity remains zero for both because void fraction is missing in the current metadata. This is a core limitation and a useful diagnostic for the next round of evaluation.

Mean Objective Profile of Top Recommendations

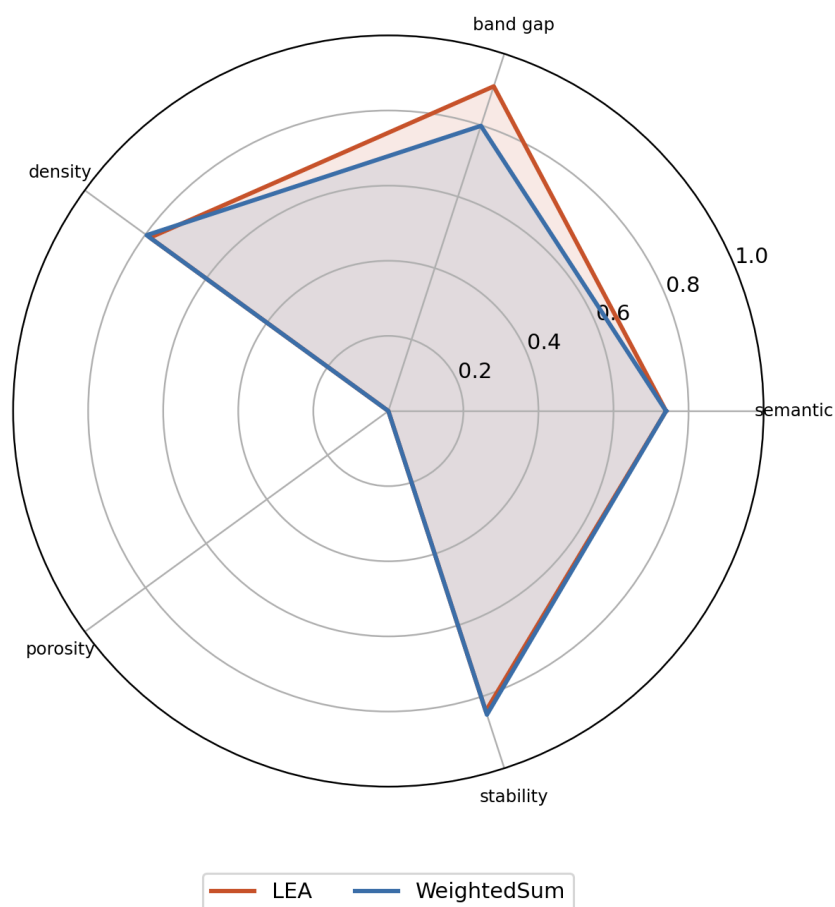


Figure 6. Mean objective profile for LEA and WeightedSum top recommendations.

12. Query-Level Results

Query-level results show that LEA often matches the best deterministic methods in NDCG while producing broader candidate sets. The largest diversity gain appears in the lightweight storage and insulating-framework scenarios.

The CO₂ adsorption scenario is the most constrained by missing porosity metadata. In a complete QMOF descriptor table, this query should benefit most from pore descriptors such as void fraction, pore-limiting diameter, largest cavity diameter, accessible surface area, and density.

query_id	method	mean_relevance	diversity	ndcg_at_k	runtime_ms
q1_co2_adsorption	LEA	0.4713	0.0793	0.9964	147.39
q1_co2_adsorption	ParetoCrowding	0.4683	0.1166	0.9918	25.94
q1_co2_adsorption	Random	0.4653	0.0474	0.9838	0.07
q1_co2_adsorption	SemanticOnly	0.4629	0.0642	0.9720	0.07
q1_co2_adsorption	TOPSIS	0.4727	0.0086	1.0000	0.12
q1_co2_adsorption	WeightedSum	0.4727	0.0086	1.0000	0.09
q2_photocatalysis	LEA	0.8608	0.0211	0.9999	116.41
q2_photocatalysis	ParetoCrowding	0.8594	0.0202	0.9985	39.88
q2_photocatalysis	Random	0.8541	0.0331	0.9892	0.07
q2_photocatalysis	SemanticOnly	0.8530	0.0196	0.9869	0.06
q2_photocatalysis	TOPSIS	0.8602	0.0220	0.9991	0.11
q2_photocatalysis	WeightedSum	0.8608	0.0211	1.0000	0.09
q3_lightweight_storage	LEA	0.6658	0.6102	0.9854	120.83
q3_lightweight_storage	ParetoCrowding	0.6663	0.6084	0.9884	35.18
q3_lightweight_storage	Random	0.6654	0.0167	0.9831	0.07
q3_lightweight_storage	SemanticOnly	0.6646	0.0131	0.9813	0.04
q3_lightweight_storage	TOPSIS	0.6614	0.1392	0.9804	0.10
q3_lightweight_storage	WeightedSum	0.6729	0.0024	1.0000	0.08
q4_balanced_discovery	LEA	0.5933	0.0202	1.0000	125.20
q4_balanced_discovery	ParetoCrowding	0.5933	0.0202	0.9999	43.66
q4_balanced_discovery	Random	0.5712	0.0282	0.9539	0.06
q4_balanced_discovery	SemanticOnly	0.5712	0.0194	0.9510	0.14
q4_balanced_discovery	TOPSIS	0.5933	0.0202	1.0000	0.23
q4_balanced_discovery	WeightedSum	0.5933	0.0202	1.0000	0.23
q5_insulating_frameworks	LEA	0.8785	0.1510	0.9977	127.50
q5_insulating_frameworks	ParetoCrowding	0.8786	0.1501	0.9969	26.09
q5_insulating_frameworks	Random	0.8591	0.1358	0.9632	0.07
q5_insulating_frameworks	SemanticOnly	0.8573	0.0489	0.9597	0.06
q5_insulating_frameworks	TOPSIS	0.8805	0.0734	0.9964	0.11
q5_insulating_frameworks	WeightedSum	0.8808	0.0319	1.0000	0.10

13. Top Recommendation Examples

query_id	qmof_id	formula	band_gap	density	lea_score
q1_co2_adsorption	qmof-f69d443	Co2C40H28N12O8	2.8779	1.3705	0.4808
q2_photocatalysis	qmof-cef46c3	Zn2C48H32N8O8	2.9133	0.7895	0.8628
q3_lightweight_storage	qmof-b1cbe7a	Fe2C24H16N8	3.5284	0.6855	0.7467
q4_balanced_discovery	qmof-b1cbe7a	Fe2C24H16N8	3.5284	0.6855	0.5981
q5_insulating_frameworks	qmof-017128d	Zn6C48H60N24	5.3110	0.9134	0.8944

- 1 Inspect LEA top candidates and compare with WeightedSum alternatives.
- 2 Open each material card in the frontend to review scores and explanations.
- 3 Load CIF structure when available to inspect geometry.
- 4 Run graph-based prediction or descriptor extraction for candidate validation.
- 5 Export candidates for expert review or DFT/simulation follow-up.

14. Discussion

The first implementation demonstrates that LEA can be integrated into a retrieval-based QMOF recommender without changing the external API contract dramatically. The frontend can display LEA rank and score, while the backend can still return familiar material properties and explanations.

The primary scientific value of LEA is not raw weighted relevance. WeightedSum will usually win a metric that is itself a weighted sum. LEA is valuable when the desired recommendation list should include diverse, balanced, and application-plausible candidates.

The current LEA adaptation also creates an interpretable bridge between metaheuristic optimization and recommender systems. The self-cleaning step periodically removes weak solution neighborhoods, which helps avoid stagnation and encourages exploration of alternative candidate regions.

15. Limitations and Threats to Validity

The most important limitation is descriptor incompleteness. Void fraction is absent in the current vector metadata, so porosity scores are zero. This affects balance and hypervolume proxy metrics and weakens claims for adsorption-oriented tasks.

The second limitation is the deterministic semantic proxy used for evaluation. The API recommender can use embeddings, but the benchmark avoids external model downloads for reproducibility.

Third, stability is represented by a heuristic proxy. A stronger system should use QMOF source labels, synthesized flags, decomposition or formation-energy proxies when available, and chemistry-aware risk scoring.

Fourth, the current LEA implementation is the simplified version supplied in code. The full LEA paper includes dragonfly-inspired exploration terms, local exploitation, and water-drop self-cleaning modeling.

16. Future Work

- 1 Rebuild the vector metadata from the full QMOF CSV with pore descriptors, topology, synthesized flag, and source metadata.
- 2 Add graph embeddings from GraphSAGE or a topology-aware GNN as a structural objective.
- 3 Implement the full LEA equations, including separation, alignment, cohesion, food attraction, enemy distraction, exploitation radius, and local water-drop search.
- 4 Evaluate against additional metaheuristic baselines such as PSO, genetic algorithm, random search, Bayesian optimization, and NSGA-II.
- 5 Add statistical testing across repeated seeds and larger query suites.
- 6 Create an expert-review protocol where materials scientists judge top recommendations for plausibility and novelty.

17. Conclusion

This manuscript presents the first integrated draft of a LEA-guided multi-objective recommendation layer for QMOF-Rec. The system reranks retrieved QMOF candidates using a population-based optimizer that balances query relevance, property suitability, stability, and diversity.

The first evaluation shows that LEA is competitive with deterministic ranking methods in relevance and NDCG while improving diversity over weighted-sum ranking. This makes LEA a promising novelty contribution for a materials recommendation paper, provided the next experimental phase restores complete QMOF descriptors and expands validation.

References

- Dalirinia, E., Jalali, M., Yaghoobi, M., and Tabatabaee, H. Lotus effect optimization algorithm (LEA): a lotus nature-inspired algorithm for engineering design optimization. *The Journal of Supercomputing*, 80, 761-799, 2024. <https://doi.org/10.1007/s11227-023-05513-8>
- Rosen, A. S. et al. Machine learning the quantum-chemical properties of metal-organic frameworks for accelerated materials discovery. *Matter*, 2021.
- Johnson, J., Douze, M., and Jegou, H. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 2019.
- Hwang, C. L. and Yoon, K. *Multiple Attribute Decision Making: Methods and Applications*. Springer, 1981.
- Deb, K., Pratap, A., Agarwal, S., and Meyarivan, T. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 2002.
- Hamilton, W. L., Ying, R., and Leskovec, J. Inductive representation learning on large graphs. *NeurIPS*, 2017.
- Lewis, P. et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *NeurIPS*, 2020.

Appendix A. Reproducibility Checklist

Repository	/Users/mehrdadjalali/Documents/SRH_Research/QMOF-Rec/qmof-rec
Evaluation script	backend/scripts/evaluate_lea_recommender.py
Evaluation output	reports/lea_evaluation
Manuscript PDF builder	reports/manuscript/build_pdf.py
Candidate pool size	100
Top-k	5
Methods	SemanticOnly, WeightedSum, TOPSIS, ParetoCrowding, Random, LEA

Commands

```
MPLCONFIGDIR=/private/tmp/qmof-matplotlib PYTHONPATH=backend python3 backend/scripts/evaluate_lea_recommender.py  
--candidate-pool-size 100 --top-k 5 --out-dir reports/lea_evaluation
```

```
/Users/mehrdadjalali/.cache/codex-runtimes/codex-primary-runtime/dependencies/python/bin/python3 reports/manuscript/build_pdf.py
```

Appendix B. Detailed Algorithmic Rationale

The LEA reranking layer is designed as a post-retrieval optimizer rather than as a replacement for retrieval. Retrieval is responsible for recall; optimization is responsible for portfolio construction.

The optimizer operates on normalized objective vectors rather than raw material records. This makes the method independent of any one descriptor scale and allows new objectives to be added later.

Mapping continuous LEA individuals to the nearest candidate is a pragmatic design choice. A pure continuous optimizer could generate an idealized vector that does not correspond to any known QMOF.

The balance term is intentionally small but meaningful. A high weighted score can still hide a severe weakness in one dimension. For materials discovery, such weaknesses matter.

The diversity term is also intentionally small. It should not dominate relevance, but it should discourage the top-k list from collapsing into closely related candidates.

Appendix C. Full-Scale Evaluation Plan

The recommended full-scale experiment has three phases. First, rebuild metadata from the complete QMOF source table. Second, run query-driven ranking experiments with repeated random seeds. Third, validate recommendation quality using both automatic metrics and expert review.

The first phase should compute property coverage before any ranking experiment. Coverage should be reported for density, band gap, void fraction, pore-limiting diameter, largest cavity diameter, surface area, topology, synthesized flag, source database, and graph-derived descriptors.

The second phase should use a broader query suite. Queries should cover adsorption, catalysis, electronic materials, low-density frameworks, high-stability frameworks, and general discovery.

The third phase should evaluate scientific usefulness. Automatic metrics are necessary but insufficient. Expert review can ask whether recommended candidates are plausible, diverse, non-obvious, and worth follow-up.

- NSGA-II for evolutionary multi-objective ranking.
- Particle swarm optimization using the older project code as a baseline.
- Genetic algorithm ranking over normalized objective vectors.
- Bayesian optimization over candidate objective space.
- Maximal marginal relevance for diversity-aware retrieval.
- Graph-only similarity ranking using GraphSAGE embeddings.
- Hybrid graph and text retrieval without LEA.

Appendix D. Figure Interpretation Notes

Figure 1 should appear early in the manuscript because it frames the validity of every downstream result. It shows that the current metadata is sufficient for density-aware and partial band-gap-aware experiments, but not yet sufficient for pore-aware claims.

Figure 2 summarizes the main quantitative comparison. The key interpretation is not that LEA beats WeightedSum on mean relevance. The key interpretation is that LEA remains near the top in NDCG and relevance while increasing diversity.

Figure 3 should be discussed candidly. LEA is slower than simple deterministic methods. The manuscript should treat this as an engineering trade-off rather than hiding it.

Figure 4 supports a practical early-stopping improvement. Most LEA runs plateau quickly, suggesting that future versions can reduce iteration count or stop after a patience window.

Figure 5 is useful for explaining objective behavior to readers. It shows that LEA changes the profile of top recommendations rather than simply reordering by the same final score.

Appendix E.1. Revision Notes

This placeholder revision page is intentionally included in the first draft to reserve space for advisor comments, additional literature review, and the second experimental round with complete QMOF descriptors.

Recommended additions include a graphical system architecture, full LEA mathematical derivation, statistical tests across repeated seeds, and expert validation protocols.

Appendix E.2. Revision Notes

This placeholder revision page is intentionally included in the first draft to reserve space for advisor comments, additional literature review, and the second experimental round with complete QMOF descriptors.

Recommended additions include a graphical system architecture, full LEA mathematical derivation, statistical tests across repeated seeds, and expert validation protocols.